Manejo de archivos en Java

Cómo escribir, leer, validar y filtrar información con archivos



Andy Anthony Jos Vale Sandra

Todo el grupo

Distribución de la exposición

Orden de exposición y tema de cada integrante

1

Andy

write() y close(), diferencia entre sobrescribir y agregar, y manejo de excepciones

2

Anthony

Escritura de archivos con FileWriter y BufferedWriter

3

Jos

Entrada, procesamiento y salida de datos al trabajar con archivos

4

Vale

Qué es el procesamiento de información, validación de datos y manejo de errores

5

Sandra

Conversión de texto a números, búsqueda y filtrado de información

Ejercicio integrador

Andy lo explica y todos participan en la demostración

Prueba · 25 min

Kahoot final · 10 min

1 Andy

write() y close()

Los dos métodos que toda escritura necesita

1 write()

Envía texto al archivo. No agrega saltos de línea por sí solo: usa \n cuando quieras una línea nueva.

2 close()

Guarda lo pendiente y libera el archivo. Si lo olvidas, parte del texto puede no llegar a escribirse.

Notas.java

```
FileWriter fw = new FileWriter("notas.txt");
fw.write("Hola, archivo!\n");
fw.write("Segunda línea\n");
fw.close();   // guarda y libera
```

Resultado dentro de notas.txt

notas.txt

```
Hola, archivo!
Segunda línea
```

1 Andy

Sobrescribir o agregar

Un solo detalle en el constructor cambia lo que pasa con el contenido anterior

Sobrescribir

Notas.java

```
FileWriter fw = new FileWriter("log.txt");
fw.write("Martes");
```

Antes

log.txt

Lunes

Después

log.txt

Martes

Agregar

Notas.java

```
FileWriter fw = new FileWriter("log.txt", true);
fw.write("Martes\n");
```

Antes

log.txt

Lunes

Después

log.txt

Lunes

Martes

Si no quieres perder lo anterior, pasa true como segundo argumento de FileWriter

1 Andy

Manejo de excepciones

Escribir en un archivo puede fallar: el programa debe saber qué hacer

Notas.java

```java
try (FileWriter fw = new FileWriter("notas.txt")) {
    fw.write("Datos guardados");
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
```

¿Qué es una IOException?

Un error de entrada o salida: la ruta no existe, no hay permiso o el archivo no se puede abrir.

`try`

Intenta ejecutar el código que puede fallar.

`catch`

Atrapa el error y responde sin que el programa se detenga.

`try ( … )`

Cierra el archivo automáticamente, como si llamara close().

2 Anthony

FileWriter

La forma más directa de escribir texto en un archivo

- Escribe caracteres directamente en el archivo.
- Crea el archivo si no existe.
- **`new FileWriter(ruta)`** sobrescribe el contenido.
- **`new FileWriter(ruta, true)`** agrega al final.

Ideal para textos cortos y pruebas rápidas

Saludo.java

```java
try (FileWriter fw = new FileWriter("saludo.txt")) {
    fw.write("Hola desde FileWriter");
} catch (IOException e) {
    e.printStackTrace();
}
```

write() escribe el texto en el archivo.

saludo.txt

```
Hola desde FileWriter
```

2 Anthony

BufferedWriter

Un búfer en memoria que junta el texto antes de escribirlo

Lista.java

```java
try (BufferedWriter bw = new BufferedWriter(new FileWriter("lista.txt"))) {
    bw.write("Línea 1");
    bw.newLine();
    bw.write("Línea 2");
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
```



	FileWriter	BufferedWriter
Mejor para	Pocos datos	Muchas líneas
Salto de línea	`"\n" escrito a mano`	`newLine()`
Se crea con	`new FileWriter(ruta)`	`new BufferedWriter(new FileWriter(ruta))`

3 Jos

Entrada, procesamiento y salida

Todo programa con archivos sigue este camino

Entrada

Leer los datos desde el archivo.

→

Procesamiento

Calcular, comparar o transformar.

→

Salida

Guardar o mostrar el resultado.

Resumen.java

```java
Scanner sc = new Scanner(new File("ventas.txt"));   // Entrada
int total = 0;
while (sc.hasNextInt()) {
    total += sc.nextInt();                          // Procesamiento
}
sc.close();
FileWriter fw = new FileWriter("resumen.txt");   // Salida
fw.write("Total: " + total);
fw.close();
```

4 Vale

Procesar y validar la información

Antes de usar un dato hay que comprobar que sirve

Procesamiento de información

Transformar los datos leídos (ordenar, calcular, convertir, filtrar) para obtener un resultado útil.

Validar antes de usar los datos

- ¿El archivo existe?
- ¿Tiene contenido?
- ¿El formato es el esperado?
- ¿El valor está dentro del rango?

Validar.java

```java
File f = new File("datos.txt");
if (!f.exists()) {
    System.out.println("No se encontró el archivo");
    return;
}
```

Errores comunes

`FileNotFoundException`	El archivo no existe o no se abre.
`IOException`	Falla al leer o escribir.
`NumberFormatException`	El texto no es un número.

5 Sandra

De texto a números

Todo lo que se lee de un archivo llega como texto





Atrapa el error

Si el texto no es un número, parseInt lanza una NumberFormatException. Siempre envuélvelo en try/catch.

Convertir.java

```java
String texto = "85";
try {
    int nota = Integer.parseInt(texto);
    System.out.println(nota + 5);   // 90
} catch (NumberFormatException e) {
    System.out.println("No es un número");
}
```

Métodos más usados

`Integer.parseInt("42")`	42
`Double.parseDouble("3.5")`	3.5

5 Sandra

Búsqueda y filtrado

Recorrer el archivo línea por línea y quedarse solo con lo que importa

notas.txt

```
Ana,92
Luis,65
Marta,78
Pedro,55
```

Notas de 70 o más

```
Ana: 92
Marta: 78
```

Filtrar.java

```
while (sc.hasNextLine()) {
    String[] p = sc.nextLine().split(",");
    int nota = Integer.parseInt(p[1].trim());
    if (nota >= 70) {
        System.out.println(p[0] + ": " + nota);
    }
}
```

Buscar por nombre

```
if (p[0].equalsIgnoreCase(buscado))
```

Compara sin importar mayúsculas.

Cómo funciona

`split(",")` separa la línea en nombre y nota.

`parseInt` convierte la nota a número.

`if (nota >= 70)` deja pasar solo los que cumplen.

Todo el grupo

Ejercicio integrador: registro de notas

Andy lo explica y todos participan en la demostración

1	**Andy**	Guardar cada nota nueva al final del archivo	`new FileWriter("notas.txt", true)`
2	**Anthony**	Escribir muchas notas seguidas usando un búfer	`BufferedWriter + newLine()`
3	**Jos**	Leer el archivo y calcular el promedio	`Scanner → suma → resumen`
4	**Vale**	Validar que el archivo exista y que cada línea tenga datos	`exists() + try/catch`
5	**Sandra**	Convertir las notas a números y filtrar los aprobados	`parseInt() + if (nota >= 70)`

Ahora les toca a ustedes

Dos momentos para practicar lo aprendido

Prueba

25 min

Kahoot final

10 min

Gracias

¿Alguna pregunta sobre el manejo de archivos?

Andy Anthony Jos Vale Sandra